HANZO THRESHOLD SIGNING

MPC Key Management for AI Infrastructure

Zach Kelling

Hanzo AI

March 2026

Abstract

We present Hanzo Threshold Signing, a system for distributed key management and cryptographic signing across Hanzo infrastructure services. The system eliminates single points of compromise for critical signing operations: IAM session tokens, KMS secret encryption keys, agent credential delegation, and ACI compute attestations. We implement two threshold signature protocols: (i) FROST (Flexible Round-Optimized Schnorr Threshold signatures) for Ed25519, used in IAM JWT signing and agent credential chains, and (ii) CGGMP21 for threshold ECDSA (secp256k1), used in blockchain transaction signing for ACI staking and Lux cross-chain operations. The system operates as a (t, n) threshold scheme where t of n signers must participate to produce a valid signature, with no single party ever holding the complete private key. We describe the distributed key generation (DKG) protocol, the signing protocol with identifiable abort, proactive key refresh without changing the public key, and integration with Hanzo KMS for share storage. Production deployment across 5 signing nodes achieves $(3, 5)$ threshold signing with median latency of 12ms for FROST and 45ms for CGGMP21, processing over 2 million threshold signatures per day.

1 Introduction

Hanzo infrastructure depends on cryptographic signing for security-critical operations:

1. **IAM session tokens**: JWT access tokens signed with Ed25519, issued by Hanzo IAM [1] and validated by every service in the platform.

2. **KMS envelope encryption**: Master keys that encrypt all secrets in Hanzo KMS [2]. Compromise of a master key exposes every secret in the system.

3. **Agent credentials**: AI agents [3] receive delegated credentials for tool access, API calls, and resource management. The signing key authorizing these delegations must not be compromised.

4. **ACI compute attestations**: GPU nodes attest to inference results by signing output hashes. These attestations are verified on-chain for the Hanzo ACI compute market [4].

In all cases, a single signing key stored on a single server is a single point of failure. If that server is compromised, the attacker can forge IAM tokens, decrypt all secrets, issue fraudulent agent credentials, or submit false compute attestations.

Threshold signatures solve this by distributing the signing key across n servers such that any t must cooperate to sign, but fewer than t servers learn nothing about the key. The combined public key is a standard Ed25519 or ECDSA public key—verifiers need no knowledge of the threshold scheme.

1.1 Protocol Selection

We use two protocols, chosen for specific properties:

Protocol	**Curve**	**Use Case**
FROST [6]	Ed25519	IAM tokens, agent credentials, API signing
CGGMP21 [7]	secp256k1	Blockchain transactions, ACI staking

Table 1: Protocol selection by use case.

FROST is preferred for Ed25519 operations due to its simplicity (2-round signing, no zero-knowledge proofs during signing). CGGMP21 is required for ECDSA because Schnorr-based threshold constructions do not produce valid ECDSA signatures.

2 FROST for Ed25519

2.1 Distributed Key Generation

FROST DKG produces key shares without any party learning the full secret key. Each participant P_i acts as both dealer and verifier in a Pedersen DKG [8]:

Definition 1 ((t, n)-Secret Sharing)**.** *A (t, n)-Shamir secret sharing of $s \in \mathbb{Z}_q$ distributes shares $s_i = f(i)$ where $f(X) = s + \sum_{j=1}^{t-1} a_j X^j$ is a random polynomial of degree $t - 1$. Any t shares determine s; fewer than t shares reveal no information about s.*

Algorithm 1 FROST Distributed Key Generation

Require: Participants $P_1, \ldots, P_n$, threshold t
Ensure: Public key Y, shares $s_1, \ldots, s_n$

```
for each participant P_i do
    Sample random polynomial f_i(X) = a_{i,0} + Σ_{j=1}^{t-1} a_{i,j} X^j
    Broadcast commitments C_{i,j} = a_{i,j} · G for j = 0, ..., t − 1
    Send share f_i(k) privately to each P_k
end for
for each participant P_k do
    Verify: f_i(k) · G ≟ Σ_{j=0}^{t-1} C_{i,j} · k^j for all i
    Compute combined share: s_k = Σ_{i=1}^{n} f_i(k)
end for
Y = Σ_{i=1}^{n} C_{i,0}                                   ▷ Combined public key
```

Theorem 1 (DKG Security)**.** *The FROST DKG protocol is secure against $t - 1$ corrupted participants under the discrete logarithm assumption on the Ed25519 curve. No coalition of fewer than t participants can reconstruct the secret key $s = \sum_i a_{i,0}$.*

2.2 Threshold Signing

FROST signing requires two rounds. In the first round, each signer generates and broadcasts a nonce commitment. In the second round, each signer produces a partial signature using their key share and the aggregated nonce.

Definition 2 (FROST Signature Share)**.** *Given message m, group nonce $R = \sum_{i \in S} R_i$, and challenge $c = H(R \| Y \| m)$, signer P_i with share s_i computes:*

$$z_i = d_i + e_i \cdot \rho_i + \lambda_i \cdot s_i \cdot c, \tag{1}$$

where (d_i, e_i) are nonce secrets, $\rho_i = H(i, m, B)$ is the binding factor, and λ_i is the Lagrange interpolation coefficient for the signing set S.

The aggregated signature $\sigma = (R, z)$ where $z = \sum_{i \in S} z_i$ is a standard Ed25519 signature verifiable with the public key Y.

Proposition 2 (Unforgeability)**.** *FROST signatures are existentially unforgeable under chosen-message attack (EUF-CMA) in the random oracle model, assuming the hardness of the discrete logarithm problem on Curve25519 [6].*

2.3 Nonce Pre-computation

To reduce signing latency, signers pre-compute nonce pairs during idle periods:

1. Each signer generates a batch of B nonce pairs (d_j, e_j) and commitments (D_j, E_j).
2. Commitments are broadcast and stored by all participants.
3. During signing, a pre-computed nonce pair is consumed. No communication is needed in the first round.

With $B = 10000$ pre-computed nonces, signing reduces to a single round with median latency of 8ms (vs. 12ms for 2-round signing).

3 CGGMP21 for ECDSA

ECDSA threshold signing is more complex than Schnorr because the ECDSA equation $s = k^{-1}(H(m) + r \cdot x)$ involves multiplication of secret values (k^{-1} and x), requiring MPC multiplication protocols.

3.1 Key Generation

CGGMP21 [7] key generation extends Feldman VSS with Paillier encryption for MPC multiplication:

1. Each party generates a Paillier key pair (N_i, ϕ_i) with $N_i > 2^{2048}$ and proves correctness via a zero-knowledge proof of safe prime product.
2. Shamir sharing of the ECDSA private key x proceeds as in FROST DKG.
3. Each party encrypts their share under their Paillier key and broadcasts the ciphertext with a range proof.

3.2 Threshold Signing Protocol

The CGGMP21 signing protocol consists of four phases:

1. **Nonce generation**: Each signer P_i samples k_i, γ_i and broadcasts commitments $\Gamma_i = \gamma_i \cdot G$ and Paillier encryptions $K_i = \texttt{Enc}(k_i)$.
2. **MPC multiplication**: Parties compute $\delta_i = k_i\gamma_j + k_j\gamma_i$ pairwise using the Paillier homomorphism, with affine-group range proofs ensuring honest behavior.
3. **Nonce reveal**: Parties reveal δ_i and reconstruct $\delta = k \cdot \gamma$. Compute $R = (\delta^{-1}) \cdot \Gamma$ where $\Gamma = \sum_i \Gamma_i$. Extract $r = R.x \bmod q$.
4. **Partial signatures**: Each party computes $\sigma_i = k_i \cdot H(m) + k_i \cdot r \cdot \chi_i$ where $\chi_i = x_i \cdot \lambda_i$ is the Lagrange-weighted share.

Theorem 3 (Identifiable Abort)**.** *If any party deviates from the CGGMP21 protocol, the honest parties can identify the cheater with probability 1 and exclude them from future signing sessions. This relies on the zero-knowledge range proofs attached to every Paillier operation.*

4 Proactive Key Refresh

Key shares must be periodically refreshed to limit the window of vulnerability if a share is compromised. Proactive secret sharing [9] re-randomizes shares without changing the public key.

Definition 3 (Key Refresh)**.** *A key refresh transforms shares* $(s_1, \ldots, s_n)$ *of secret* s *into fresh shares* $(s'_1, \ldots, s'_n)$ *of the same secret* s*, such that knowing* s_i *and* s'_j *(for different epochs) provides no advantage in recovering* s*.*

The protocol:

1. Each party P_i generates a random polynomial $g_i(X)$ of degree $t-1$ with $g_i(0) = 0$.
2. P_i sends $g_i(k)$ privately to each P_k.
3. Each party computes $s'_k = s_k + \sum_i g_i(k)$.

Since $\sum_i g_i(0) = 0$, the secret is unchanged: $s' = s$. But shares are fully re-randomized: knowledge of old shares provides no information about new shares.

Hanzo infrastructure performs key refresh every 24 hours. Refresh completes in <100ms and does not interrupt signing operations.

5 Integration with Hanzo Services

5.1 IAM Token Signing

Hanzo IAM [1] issues JWT tokens signed with Ed25519. The IAM signing key is split across 5 threshold signing nodes in a $(3, 5)$ configuration:

1. User authenticates via OIDC (password, passkey, or SSO).
2. IAM constructs the JWT payload (claims, expiry, org scope).
3. IAM sends the signing request to the threshold signing service.
4. Three of five signing nodes produce partial signatures; the coordinator aggregates them into a standard Ed25519 signature.
5. The JWT is returned to the user. Verifiers use the public key from the IAM JWKS endpoint—no knowledge of threshold signing needed.

5.2 KMS Master Key Protection

Hanzo KMS [2] uses envelope encryption: data encryption keys (DEKs) are encrypted with a master key (KEK). The KEK is threshold-shared:

- KEK encryption uses threshold ECDH: each signing node holds a share of the ECDH private key, and t nodes must cooperate to derive the shared secret used for AES-256-GCM encryption of the DEK.
- KEK rotation is a key refresh operation—existing DEKs encrypted under the old KEK are re-encrypted under the refreshed key without exposing plaintext DEKs.

5.3 Agent Credential Signing

AI agents [3] receive delegated credentials via certificate chains:

1. The platform root key (threshold-shared) signs an org-level intermediate certificate.
2. The org intermediate signs an agent-specific credential with constrained permissions (allowed tools, API scopes, time bound).
3. Agents present the credential chain when accessing resources. Each service verifies the chain back to the well-known root public key.

5.4 ACI Compute Attestation

ACI GPU nodes [4] sign inference attestations with threshold ECDSA:

- Each GPU node in a verification group holds a share of the group signing key.
- After verifying an inference result, t of n verifiers produce a threshold ECDSA signature over the result hash.
- The signature is submitted on-chain as proof of verified computation.

6 Security Analysis

Theorem 4 (Threshold Security)**.** *The Hanzo Threshold Signing system with* $(t, n) = (3, 5)$ *provides security under the following adversary model: an adversary who compromises up to* $t - 1 = 2$ *signing nodes cannot forge signatures, recover the private key, or distinguish partial signatures from random values.*

Remark 1. *The identifiable abort property of CGGMP21 ensures that a compromised node cannot cause signing to fail silently. If a node submits invalid partial signatures, the coordinator detects the fault, identifies the malicious node, and re-runs the protocol with the remaining honest nodes (provided* $\geq t$ *honest nodes remain).*

6.1 Deployment Topology

The 5 signing nodes are deployed across 3 geographic regions and 2 cloud providers:

- 2 nodes in US (DOKS lux-k8s, separate availability zones).
- 2 nodes in EU (DOKS lux-k8s, separate availability zones).
- 1 node in APAC (DOKS lux-k8s).

An adversary must compromise nodes in at least 2 geographic regions to reach the threshold.

7 Performance

Operation	Protocol	Median (ms)	p99 (ms)
DKG (one-time)	FROST	85	120
DKG (one-time)	CGGMP21	2400	3100
Signing	FROST	12	28
Signing (pre-computed nonce)	FROST	8	18
Signing	CGGMP21	45	95
Key refresh	FROST	35	60
Key refresh	CGGMP21	180	310

Table 2: Latency for $(3, 5)$ threshold operations across 3 geographic regions. Measured over 30 days of production traffic.

The system processes over 2 million threshold signatures per day (predominantly FROST for IAM tokens), with 99.97% availability over the measurement period.

8 Related Work

FROST [6] provides the theoretical foundation for our Ed25519 threshold scheme. We extend it with pre-computed nonce batching for latency-sensitive IAM signing.

CGGMP21 [7] improves upon GG20 [10] with identifiable abort and reduced communication complexity. We adopt CGGMP21 for all ECDSA operations.

Lux Network [5] uses threshold signatures for validator set management and cross-chain bridge security. Hanzo Threshold Signing reuses the same FROST implementation for infrastructure signing.

Fireblocks MPC [11] provides commercial MPC wallet infrastructure. Unlike Fireblocks, Hanzo Threshold Signing is purpose-built for AI infrastructure signing (IAM, KMS, agent credentials), not financial custody.

9 Conclusion

Hanzo Threshold Signing eliminates single points of compromise across Hanzo infrastructure by distributing signing keys via $(3, 5)$ threshold schemes. FROST for Ed25519 achieves 8–12ms signing latency for IAM tokens and agent credentials. CGGMP21 for ECDSA achieves 45ms signing latency for blockchain operations. Proactive key refresh every 24 hours limits exposure from share compromise. The system processes 2M+ daily threshold signatures with 99.97% availability.

References

[1] Hanzo AI Research. Hanzo IAM: Identity and access management for AI platforms. *Hanzo AI Research*, 2024.

[2] Hanzo AI Research. Hanzo KMS: Secrets management with per-environment rotation and audit trail. *Hanzo AI Research*, September 2021.

[3] Hanzo AI Research. Hanzo Agent SDK: Framework for building AI agents. *Hanzo AI Research*, 2025.

[4] Hanzo AI Research. Hanzo ACI: AI Chain Infrastructure for decentralized compute markets. *Hanzo AI Research*, December 2024.

[5] Lux Partners. Lux: A scalable multi-consensus blockchain with post-quantum cryptography. *Lux Network Technical Report*, 2024.

[6] C. Komlo and I. Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In *Proc. SAC*, 2020.

[7] R. Canetti, R. Gennaro, S. Goldfeder, N. Makriyannis, and U. Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In *Proc. CCS*, 2020.

[8] T. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Proc. CRYPTO*, 1991.

[9] A. Herzberg, S. Jarecki, H. Krawczyk, and M. Yung. Proactive secret sharing, or: How to cope with perpetual leakage. In *Proc. CRYPTO*, 1995.

[10] R. Gennaro and S. Goldfeder. One round threshold ECDSA with identifiable abort. *IACR Cryptology ePrint Archive*, 2020/540.

[11] Fireblocks. MPC wallet infrastructure for digital assets. *Fireblocks Documentation*, 2023.